

CSC10330 计算机系统导论

x64 速查表

Fall 2019

1. x64 寄存器 (x64 Registers)

x64 汇编代码使用 16 个 64 位寄存器。此外，其中一些寄存器的低位字节可以独立作为 32 位、16 位或 8 位寄存器进行访问。寄存器名称如下：

8字节寄存器 (64-bit)	字节 0-3 (32-bit)	字节 0-1 (16-bit)	字节 0 (8-bit)
%rax	%eax	%ax	%al
%rcx	%ecx	%cx	%cl
%rdx	%edx	%dx	%dl
%rbx	%ebx	%bx	%bl
%rsi	%esi	%si	%sil
%rdi	%edi	%di	%dil
%rsp	%esp	%sp	%spl
%rbp	%ebp	%bp	%bpl
%r8	%r8d	%r8w	%r8b
%r9	%r9d	%r9w	%r9b
%r10	%r10d	%r10w	%r10b
%r11	%r11d	%r11w	%r11b
%r12	%r12d	%r12w	%r12b
%r13	%r13d	%r13w	%r13b
%r14	%r14d	%r14w	%r14b
%r15	%r15d	%r15w	%r15b

有关寄存器使用的更多详细信息，请参见下文的“寄存器使用”部分。

2. 操作数指示符 (Operand Specifiers)

基本类型的操作数指示符如下。在下表中：

- **Imm** 指的是常数值，例如 `0x8048d8e` 或 `48`。
- E_x 指的是寄存器，例如 `%rax`。
- $R[E_x]$ 指的是存储在寄存器 E_x 中的值。
- $M[x]$ 指的是存储在内存地址 x 处的值。

类型	格式 (From)	操作数值 (Operand Value)	名称
立即数 (Immediate)	$\$Imm$	Imm	立即数
寄存器 (Register)	E_a	$R[E_a]$	寄存器寻址
存储器 (Memory)	Imm	$M[Imm]$	绝对寻址
存储器 (Memory)	(E_a)	$M[R[E_a]]$	间接寻址
存储器 (Memory)	$Imm(E_b, E_i, s)$	$M[Imm + R[E_b] + (R[E_i] \times s)]$	比例变址寻址 (Scaled indexed)

关于操作数指示符的更多信息可以在教科书的第 169-170 页找到。

3. x64 指令 (x64 Instructions)

在下表中：

- "byte" 指的是单字节整数（后缀 **b**）。
- "word" 指的是双字节整数（后缀 **w**）。
- "doubleword" 指的是四字节整数（后缀 **d**）。
- "quadword" 指的是八字节值（后缀 **q**）。

大多数指令（如 `mov`）使用后缀来表示操作数的大小。例如，将一个四字（quadword）从 `%rax` 移动到 `%rbx` 的指令是 `movq %rax, %rbx`。

有些指令（如 `ret`）不需要后缀。其他指令（如 `movs` 和 `movz`）使用两个后缀，因为它们将第一个后缀类型的操作数转换为第二个后缀类型。因此，将 `%al` 中的字节转换为 `%ebx` 中的双字并进行零扩展的汇编代码是 `movzbl %al, %ebx`。

在下表中，除非另有说明，指令均使用一个后缀。

3.1 数据传送 (Data Movement)

指令	描述	页码
单后缀指令		
<code>mov S, D</code>	将源 (S) 移动到目的 (D)	171
<code>push S</code>	将源 (S) 压入栈	171
<code>pop D</code>	将栈顶弹出至目的 (D)	171

指令	描述	页码
双后缀指令		
<code>movs S, D</code>	将字节移动到字/双字/四字（符号扩展）	171
<code>movz S, D</code>	将字节移动到字/双字/四字（零扩展）	171
无后缀指令		
<code>cwtl</code>	将 <code>%ax</code> 中的字转换为 <code>%eax</code> 中的双字（符号扩展）	182
<code>cltq</code>	将 <code>%eax</code> 中的双字转换为 <code>%rax</code> 中的四字（符号扩展）	182
<code>cqto</code>	将 <code>%rax</code> 中的四字转换为 <code>%rdx:%rax</code> 中的八字（符号扩展）	182

3.2 算术运算 (Arithmetic Operations)

除非另有说明，所有算术运算指令均有一个后缀。

3.2.1 一元运算 (Unary Operations)

指令	描述	页码
<code>inc D</code>	加 1	178
<code>dec D</code>	减 1	178
<code>neg D</code>	算术取负	178
<code>not D</code>	按位取反	178

3.2.2 二元运算 (Binary Operations)

指令	描述	页码
<code>leaq S, D</code>	加载源 (S) 的有效地址到目的 (D)	178
<code>add S, D</code>	将源 (S) 加到目的 (D)	178
<code>sub S, D</code>	从目的 (D) 中减去源 (S)	178
<code>imul S, D</code>	目的 (D) 乘以源 (S)	178
<code>xor S, D</code>	目的 (D) 与源 (S) 按位异或	178
<code>or S, D</code>	目的 (D) 与源 (S) 按位或	178
<code>and S, D</code>	目的 (D) 与源 (S) 按位与	178

3.2.3 移位运算 (Shift Operations)

指令	描述	页码
<code>sal / shl k, D</code>	目的 (D) 左移 k 位	179
<code>sar k, D</code>	目的 (D) 算术右移 k 位	179
<code>shr k, D</code>	目的 (D) 逻辑右移 k 位	179

3.2.4 特殊算术运算 (Special Arithmetic Operations)

指令	描述	页码
<code>imulq S</code>	<code>%rax</code> 乘以 S 的有符号全乘。结果存储在 <code>%rdx:%rax</code>	182
<code>mulq S</code>	<code>%rax</code> 乘以 S 的无符号全乘。结果存储在 <code>%rdx:%rax</code>	182
<code>idivq S</code>	有符号除法 <code>%rdx:%rax</code> 除以 S。商存 <code>%rax</code> ，余数存 <code>%rdx</code>	182
<code>divq S</code>	无符号除法 <code>%rdx:%rax</code> 除以 S。商存 <code>%rax</code> ，余数存 <code>%rdx</code>	182

3.3 比较和测试指令 (Comparison and Test Instructions)

比较指令也有一个后缀。

指令	描述	页码
<code>cmp S2, S1</code>	根据 $S_1 - S_2$ 设置条件码	185
<code>test S2, S1</code>	根据 $S_1 \& S_2$ 设置条件码	185

3.4 访问条件码 (Accessing Condition Codes)

以下指令均无后缀。

3.4.1 条件设置指令 (Conditional Set Instructions)

指令	描述	条件码	页码
<code>sete / setz D</code>	相等/为零时设置	ZF	187
<code>setne / setnz D</code>	不相等/非零时设置	~ZF	187
<code>sets D</code>	负数时设置	SF	187
<code>setns D</code>	非负时设置	~SF	187
<code>setg / setnle D</code>	大于时设置 (有符号)	<code>~(SF^OF) & ~ZF</code>	187
<code>setge / setnl D</code>	大于等于时设置 (有符号)	<code>~(SF^OF)</code>	187

指令	描述	条件码	页码
<code>setl / setnge D</code>	小于时设置 (有符号)	<code>SF^OF</code>	187
<code>setle / setng D</code>	小于等于时设置	<code>(SF^OF) ZF</code>	187
<code>seta / setnbe D</code>	高于时设置 (无符号)	<code>~CF & ~ZF</code>	187
<code>setae / setnb D</code>	高于等于时设置 (无符号)	<code>~CF</code>	187
<code>setb / setnae D</code>	低于时设置 (无符号)	<code>CF</code>	187
<code>setbe / setna D</code>	低于等于时设置 (无符号)	<code>CF ZF</code>	187

3.4.2 跳转指令 (Jump Instructions)

指令	描述	条件码	页码
<code>jmp Label</code>	跳转到标签		189
<code>jmp *Operand</code>	跳转到指定位置		189
<code>je / jz Label</code>	相等/为零时跳转	<code>ZF</code>	189
<code>jne / jnz Label</code>	不相等/非零时跳转	<code>~ZF</code>	189
<code>js Label</code>	负数时跳转	<code>SF</code>	189
<code>jns Label</code>	非负时跳转	<code>~SF</code>	189
<code>jg / jnle Label</code>	大于时跳转 (有符号)	<code>~(SF^OF) & ~ZF</code>	189
<code>jge / jnl Label</code>	大于等于时跳转 (有符号)	<code>~(SF^OF)</code>	189
<code>jil / jnge Label</code>	小于时跳转 (有符号)	<code>SF^OF</code>	189
<code>jle / jng Label</code>	小于等于时跳转	<code>(SF^OF) ZF</code>	189
<code>ja / jnbe Label</code>	高于时跳转 (无符号)	<code>~CF & ~ZF</code>	189
<code>jae / jnb Label</code>	高于等于时跳转 (无符号)	<code>~CF</code>	189
<code>jb / jnae Label</code>	低于时跳转 (无符号)	<code>CF</code>	189
<code>jbe / jna Label</code>	低于等于时跳转 (无符号)	<code>CF ZF</code>	189

3.4.3 条件传送指令 (Conditional Move Instructions)

条件传送指令没有任何后缀，但源操作数和目的操作数的大小必须相同。

指令	描述	条件码	页码
<code>cmove / cmovz S, D</code>	相等/为零时传送	<code>ZF</code>	206

指令	描述	条件码	页码
<code>cmove / cmovnz S, D</code>	不相等/非零时传送	<code>~ZF</code>	206
<code>cmovs S, D</code>	负数时传送	<code>SF</code>	206
<code>cmovns S, D</code>	非负时传送	<code>~SF</code>	206
<code>cmovg / cmovnl S, D</code>	大于时传送 (有符号)	<code>~(SF^OF) & ~ZF</code>	206
<code>cmovge / cmovnl S, D</code>	大于等于时传送 (有符号)	<code>~(SF^OF)</code>	206
<code>cmovl / cmovnge S, D</code>	小于时传送 (有符号)	<code>SF^OF</code>	206
<code>cmovle / cmovng S, D</code>	小于等于时传送	<code>(SF^OF) ZF</code>	206
<code>cmove / cmovnbe S, D</code>	高于时传送 (无符号)	<code>~CF & ~ZF</code>	206
<code>cmovae / cmovnb S, D</code>	高于等于时传送 (无符号)	<code>~CF</code>	206
<code>cmovb / cmovnae S, D</code>	低于时传送 (无符号)	<code>CF</code>	206
<code>cmovbe / cmovna S, D</code>	低于等于时传送 (无符号)	<code>CF ZF</code>	206

3.5 过程调用指令 (Procedure Call Instruction)

过程调用指令没有任何后缀。

指令	描述	页码
<code>call Label</code>	压入返回地址并跳转到标签	221
<code>call *operand</code>	压入返回地址并跳转到指定位置	221
<code>leave</code>	将 <code>%rsp</code> 设置为 <code>%rbp</code> ，然后将栈顶弹出至 <code>%rbp</code>	221
<code>ret</code>	从栈中弹出返回地址并跳转到该地址	221

4. 编码实践 (Coding Practices)

4.1 注释 (Commenting)

你编写的每个函数都应该在开头有一个注释，描述该函数的功能及其接受的参数。此外，我们强烈建议在汇编代码旁边加上注释，用伪代码或某种高级语言说明每组指令的作用。换行符也有助于将语句分组为逻辑块，以提高可读性。

4.2 数组 (Arrays)

数组在内存中存储为连续的数据块。通常，数组变量充当指向内存中数组第一个元素的指针。要访问给定的数组元素，需将索引值乘以元素大小，然后加到数组指针上。例如，如果 `arr` 是一个 `int` 数组，语句：

```
arr[i] = 3;
```

可以用 x86-64 表示如下（假设 `arr` 的地址存储在 `%rax` 中，索引 `i` 存储在 `%rcx` 中）：

```
movq $3, (%rax, %rcx, 8)
```

关于数组的更多信息可以在教科书的第 232-241 页找到。

4.3 寄存器使用 (Register Usage)

x86-64 中有 16 个 64 位寄存器: `%rax`, `%rbx`, `%rcx`, `%rdx`, `%rdi`, `%rsi`, `%rbp`, `%rsp`, 和 `%r8-r15`。

- 其中, `%rax`, `%rcx`, `%rdx`, `%rdi`, `%rsi`, `%rsp`, 和 `%r8-r11` 被认为是**调用者保存 (caller-save)** 寄存器, 这意味着它们不一定在函数调用之间被保存。
- 按照惯例, `%rax` 用于存储函数的返回值 (如果存在且长度不超过 64 位)。(像结构体这样较大的返回类型使用栈返回。)
- 寄存器 `%rbx`, `%rbp`, 和 `%r12-r15` 是**被调用者保存 (callee-save)** 寄存器, 这意味着它们在函数调用之间被保存。
- 寄存器 `%rsp` 用作栈指针, 指向栈中的最顶层元素。
- 此外, `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, 和 `%r9` 用于向被调用的函数传递前六个整数或指针参数。额外的参数 (或按值传递的大型参数如结构体) 在栈上传递。

在 32 位 x86 中, 基址指针 (以前是 `%ebp`, 现在是 `%rbp`) 用于跟踪当前栈帧的基址, 被调用的函数在将基址指针更新为其自己的栈帧之前, 会保存调用者的基址指针。随着 64 位架构的出现, 这种情况大多被消除了, 除了少数特殊情况, 即编译器无法提前确定需要为特定函数分配多少栈空间时 (参见动态栈分配)。

4.4 栈组织和函数调用 (Stack Organization and Function Calls)

4.4.1 调用函数 (Calling a Function)

要调用一个函数, 程序应将前六个整数或指针参数放入寄存器 `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, 和 `%r9` 中; 随后的参数 (或大于 64 位的参数) 应压入栈中, 第一个参数在最上面。然后, 程序应执行 `call` 指令, 该指令将返回地址压入栈中并跳转到指定函数的开始处。

示例:

```
# 调用 foo(1, 15)
movq $1, %rdi    # 将 1 移入 %rdi
movq $15, %rsi   # 将 15 移入 %rsi
call foo         # 压入返回地址并跳转到标签 foo
```